

Compiler Working Group: 2026-08-06

Michael Ferguson

Agenda / Topics

Meeting Timing: Will switch to Tuesday 9AM Eastern, need to ask Quantinuum if they can send someone

Summary of OpenQSE and Compiler Working Group Scope

Summary of Working Group Goals

Stages of FTQC compilation

Invitation to present at a future meeting

Discussion of Survey Paper Outline

Michael has to leave at half past

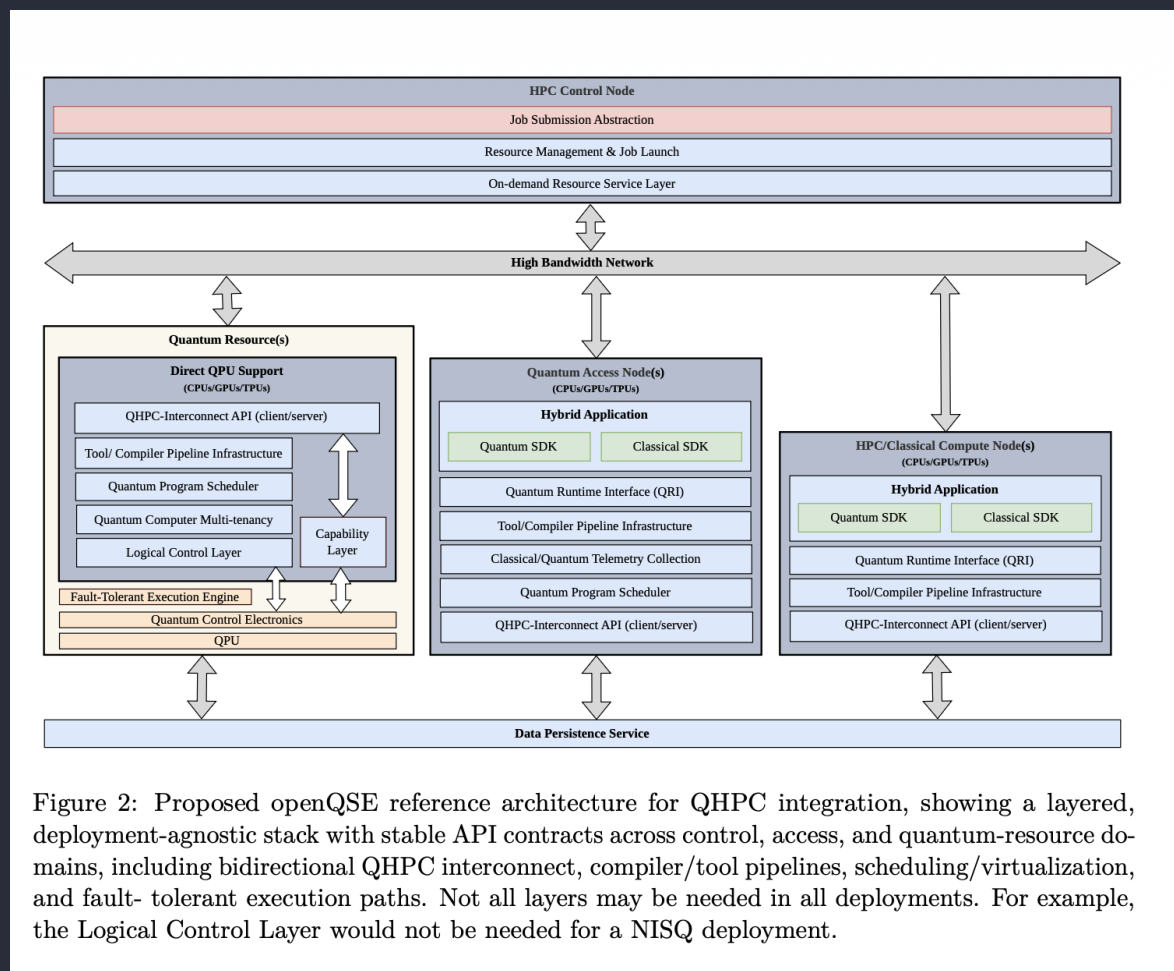
OpenQSE

OpenQSE is specifying a complete software stack for QHPC integration

OpenQSE is a community effort and includes working groups focused on specifying interfaces:

- Software Architecture
- System Architecture and Workflows
- Quantum Resource Management
- Control Electronics
- Compiler (us!)

Compiler WG is unique in its need to focus on FTQC and inability to abstract away circuit representation



<https://arxiv.org/abs/2604.20912v2>

Scope & Focus: What this workstream is responsible for

The compilation layers in the openQSE reference architecture including definition of OpenQSE-preferred circuit formats and providing for translation among circuit formats in order to connect SDKs to QPUs in a portable manner.

In-scope:

Specify an OpenQSE middleware compiler component, including interfaces to software from other workstreams:

- Quantum Resource Interface (QRI) interactions with the compiler (device queries, interaction with device compiler, circuit interchange format)
- Workflow/Application (System Architecture and Workflows WG) interactions with compiler (SDK adapters)
- Runtime layer (SW Arch WG) interactions with the compiler, if any

Develop reference implementations of such a compiler

Adapters that connect quantum SDKs to the OpenQSE compiler

Circuit compilation and translation that supports the above

Making choices as to preferred compiler IR and circuit interchange formats

<https://openqse.atlassian.net/wiki/spaces/oQSEO/pages/121405469/Compiler+Infrastructure+Working+Group>

Scope Decisions from OpenQSE Day

Scope will include FTQC/QEC

Scope will include handling hybrid quantum-classical circuits

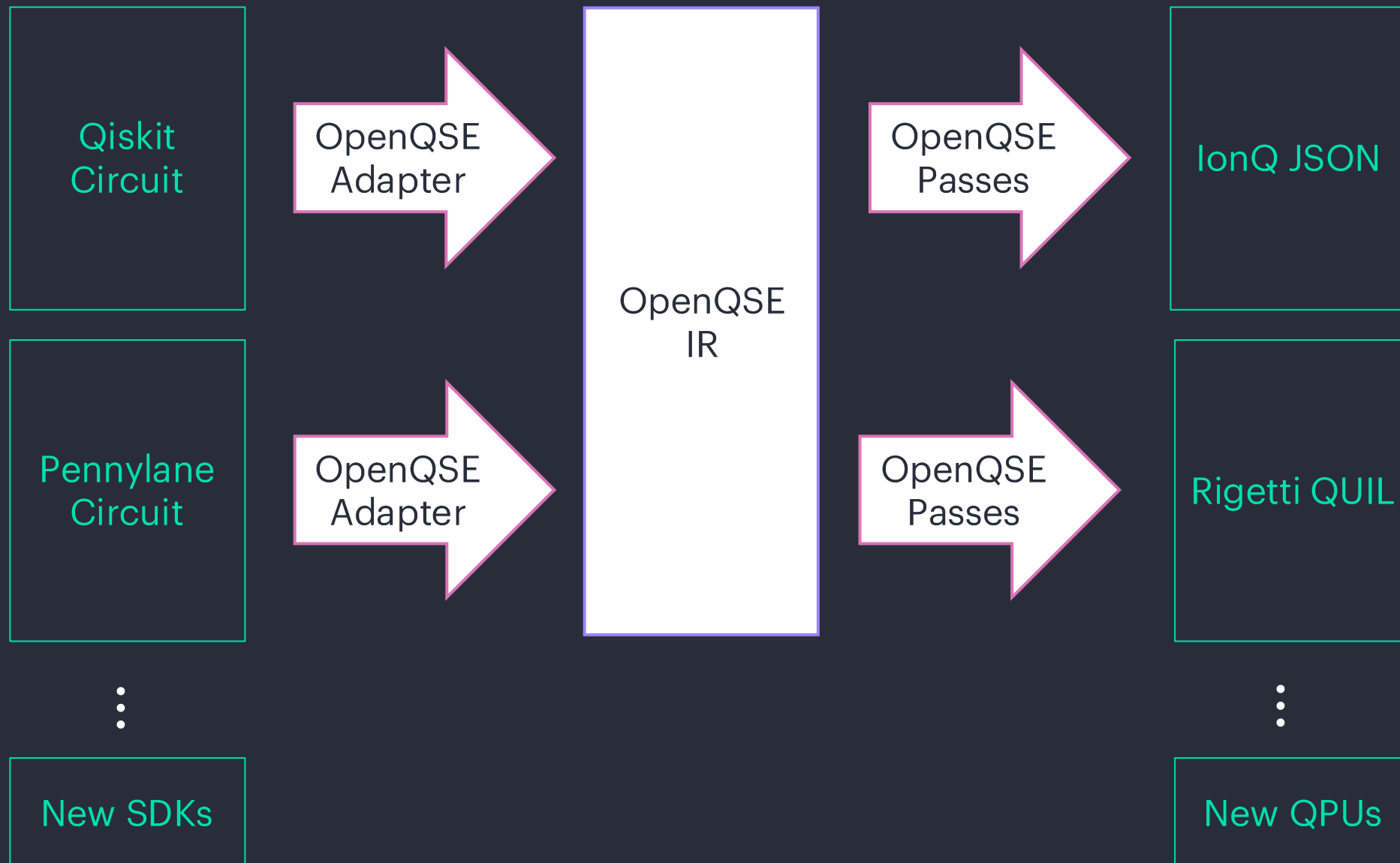
We expect that algorithms/applications will need to customize compilation

- Use a coarse-grain idea of compilation stages and allow plugins to replace one or more stages or add passes at the end of a stage

Define an OpenQSE preferred compiler IRs

- We expect multiple IRs will be required, especially for FTQC
- Focus on making preferred IRs when they are relevant for interfaces

Define an OpenQSE preferred interchange format





Probably handles:

- Specifying circuits

May handle:

- Compiling to native gates
- User-controllable transformation passes

Probably handles:

- Translation between circuit interchange formats

May handle:

- Compiling to native gates
- Mapping
- QEC compilation
- User-controllable transformations

Probably handles:

- Compiling to native gates
- Mapping
 - (with calibration data)
- QPU-specific optimization

May handle:

- QEC compilation
 - (optimized for QPU)

Working Group Goals

What the SDK does and what the QPU infrastructure does depend on the specifics!

Need flexibility to integrate at different stages of FTQC compilation

Overall goals of the working group

1. Specifying interfaces for compiler components
2. Providing a reference implementation that fills in gaps and can be a community resource

Near-term goals:

In 6 months, a survey paper, focused on FTQC compilation

A key outcome of compiler WG is to identify the primary phases of FTQC compilation and the likely integration points where different QPUs will plug in.

Adam M. Notes a key gap: interchange format for driving control electronics as well as controlling decoder: need something that combines STIM observables and OpenQASM 3 properties

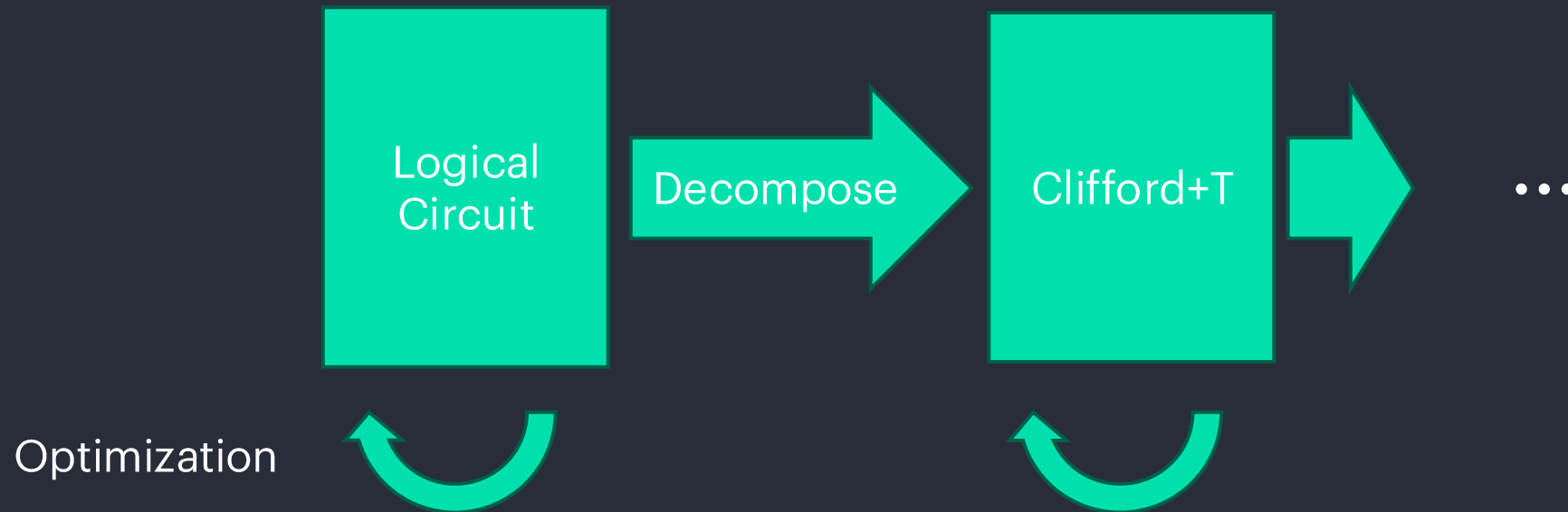
The Role of the OpenQSE Compiler Reference Implementation

I am proposing two roles for the OpenQSE compiler effort:

1. Provide circuit translation needed for portability & specify interfaces
 - (e.g., translating from OpenQASM 3 to QUIL, ...)
 - Connect circuit compilers in SDKs to QPU-specific circuit compilers
2. Create a community around compiler passes that can be reused by different vendors

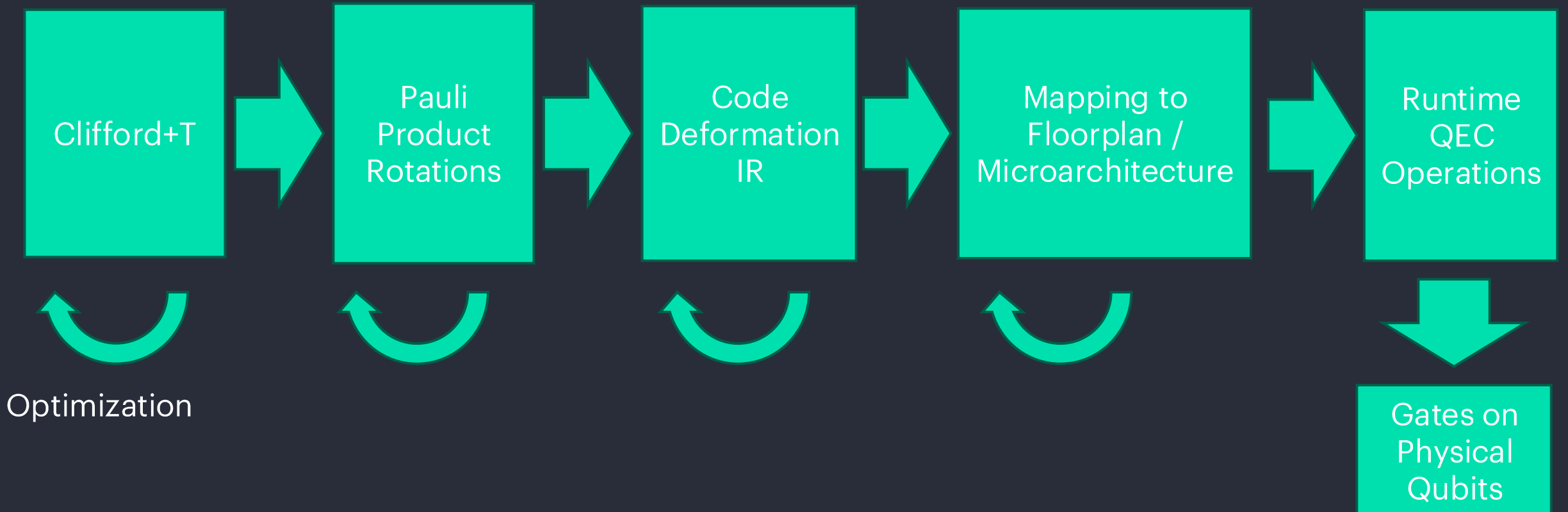
What are the stages of FTQC compilation?

FTQC Compiler Sketch: Starting Point



FTQC Compiler Sketch: Stationary Qubits

“micro ops”
probably in HW



Optimization Technique: PPR

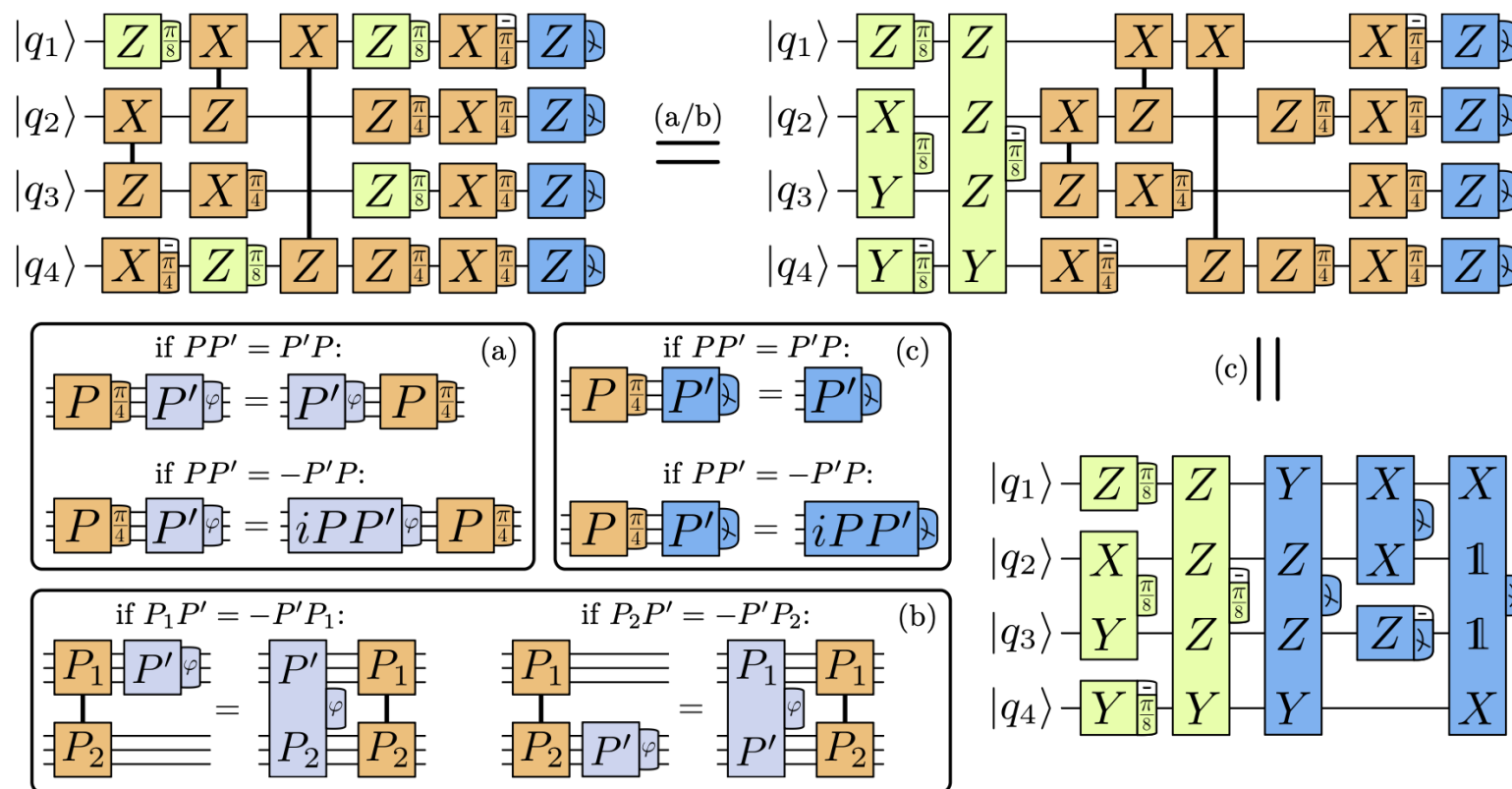


Figure 4: A generic circuit consists of $\pi/4$ rotations (orange), $\pi/8$ rotations (green) and measurements (blue). The Pauli product in each box specifies the axis of rotation or the basis of measurement. If the Pauli operator is $-P$ instead of P , a minus sign is found in the corner of the box, such that, e.g., $Z_{-\pi/4}$ corresponds to an $S^\dagger$ gate. Using the commutation rules in (a/b), all Clifford gates can be moved to the end of the circuit. Using (c), the Clifford gates can be absorbed by the final measurements.

Q: Is PPR capable of being generalized to hybrid quantum-classical programs?

How to handle mid-circuit measurements with PPR?

Conditional gates?

Loops or other control flow?

FTQC Floorplan

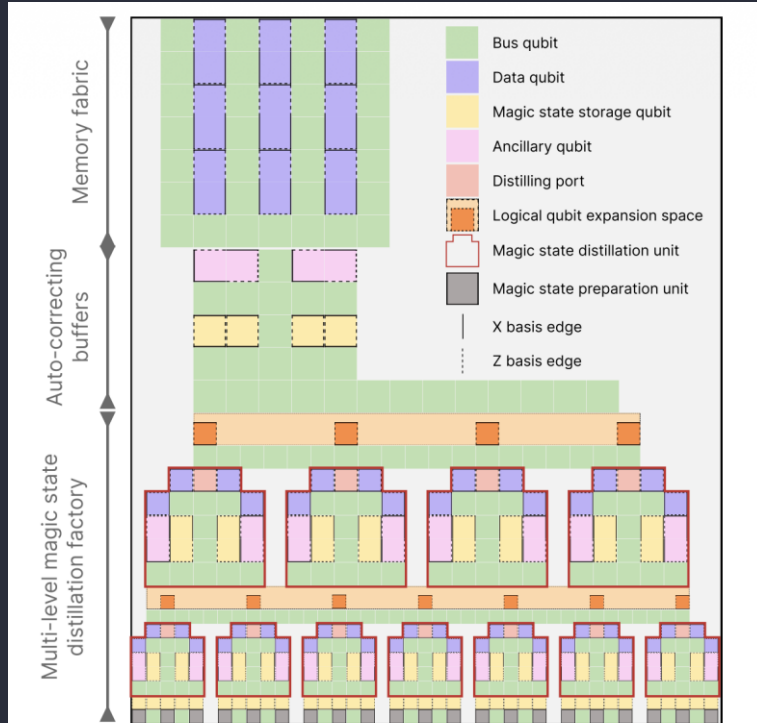


Figure 16: Example of a logical layout of a modular fault-tolerant quantum architecture of the sort designed by TopQAD [78, 79, 177]. In this example, two distillation levels are used, composed of four and two distillation units each from lowest to highest.

[Mohseni et al., How to Build a Quantum Supercomputer: Scaling from Hundreds to Millions of Qubits](#)

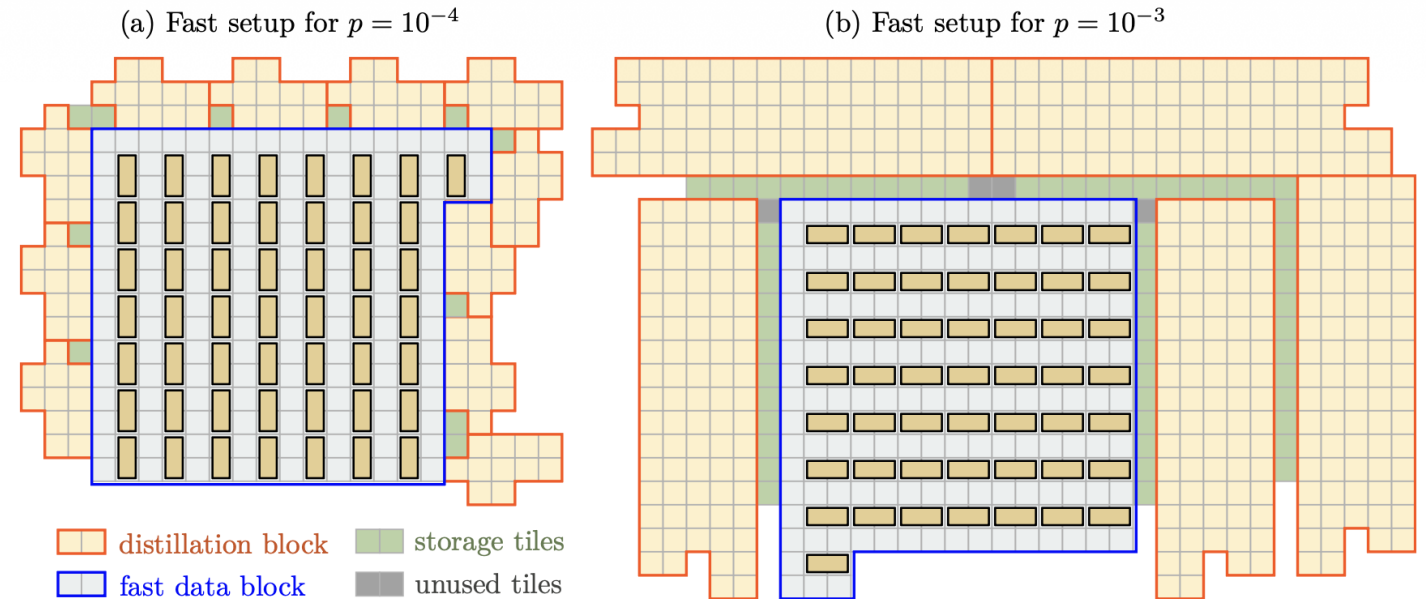
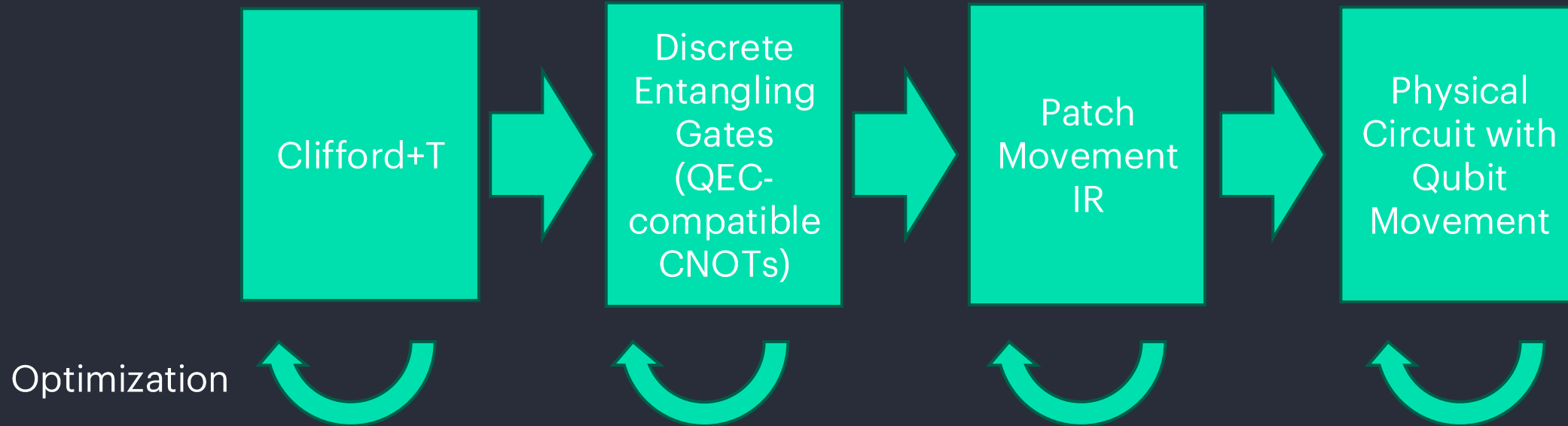


Figure 23: Fast setups using fast data blocks and 11 15-to-1 distillation blocks for $p = 10^{-4}$ or 5 116-to-12 distillation block for $p = 10^{-3}$.

[Daniel Litinski, A Game of Surface Codes: Large-Scale Quantum Computing with Lattice Surgery](#)

FTQC Compiler Sketch: Moving Qubits



qLDPC codes represent multiple logical qubits per patch and moving qubits allow transversal CNOTs but not CNOTs within a patch

Present to us in a future meeting!

Do the stages described match systems that you know about?

Can you present to tell us about it?

Matthias: plans to
present in 4 weeks on
Tuesday (3rd Sept)

What phases of FTQC compilation do you see as required?

Riverlane in
future:
dialects for
circuits, QECi
interface

What FTQC IRs are candidates for our preferred IRs?

Survey Paper

- FTQC compilation concerns: what are the conceptual circuit representations required at different stages?
 - Describe proposed frameworks
 - Describe what we envision as a general framework
- Understand existing quantum compilers; what IRs they use; what passes they perform
- Define criteria to select an IR
- Include a chart about quantum-classical capabilities
- Is there convergence on IRs? Testing?
- Performance and capabilities in terms of circuit size; how do tools scale with qubit and gate count?
- *Next step: develop an outline; Michael can create an Overleaf*

Missing interchange format
describing decoding and
hybrid circuits

How do we know that FTQC compilation
will scale / interchange formats will not use
too much data; do these need to be
serialized or binary